Android_OTA 升级量产问题排查指南

版本号: V1.2

发布日期: 2025.08.30

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
V1.0	2023.3.20	AWA1830	正式版本
V1.1	2023.11.15	AWA1633	添加 OTA 升级失败排查实例
V1.2	2025.08.30	AWA2315	更新文档语法错误

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 前言 **1**
1.1 概述 1
1.2 读者对象 1

2 OTA 问题分析手段 **2**
2.1 常见的 OTA 问题分类 2
2.2 OTA 问题调试手段 2
2.3 OTA 问题 log 分析 3

3 OTA 打包问题 **5**
3.1 OTA 打包命令实现原理 5
3.2 OTA 包大于 2G，打包失败 6
3.2.1 问题描述 6
3.2.2 问题分析 6
3.2.3 问题解决方案 6

4 update_engine 系统升级问题 **8**
4.1 update_engine 系统升级流程 8
4.2 OTA 签名问题 8
4.2.1 问题描述 8
4.2.2 问题分析 8
4.2.3 问题原因 10
4.2.4 问题解决方案 10
4.3 OTA 差分包版本不一致问题 11
4.3.1 问题描述 11
4.3.2 问题分析 11
4.3.3 问题解决方案 12
4.4 OTA 包损坏问题 12
4.4.1 问题描述 12
4.4.2 问题分析 12
4.4.3 问题解决方案 13
4.5 remount 之后 OTA 升级失败问题 13
4.5.1 问题描述 13
4.5.2 问题分析 13
4.5.3 问题解决方案 14
4.6 dragonface 修改固件之后，ota 升级失败 14
4.6.1 问题描述 14
4.6.2 问题分析 14
4.6.3 问题原因 15

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.6.4 问题解决方案 15

5 升级后系统重启失败问题 16

5.1 升级后系统重启流程 16

5.2 OTA 升级后，多次断电启动，版本回退至 OTA 前版本 17

5.2.1 问题描述 17

5.2.2 问题分析 17

5.2.3 问题解决方案 17

5.3 ota 包大于 2G 的时候 OTA 升级失败问题 17

5.3.1 问题描述 17

5.3.2 问题分析 18

5.3.3 问题原因 18

5.3.4 问题解决方案 18

5.4 修改分区表，OTA 升级后系统启动失败 25

5.4.1 问题描述 25

5.4.2 问题分析 25

5.4.3 问题解决方案 26

5.5 修改 fstab 文件，OTA 升级后系统启动失败 26

5.5.1 问题描述 26

5.5.2 问题分析 26

5.5.3 问题解决方案 26

6 FAQ 27

6.1 ota 更新 SettingsProvider 数据库 27

6.2 如何支持 ota 升级到旧版本系统 27

6.3 升级注意事项 28

6.3.1 OTA 不能改变分区数目及其大小以及 fstab 参数 28

6.3.2 Recovery 升级 28

6.4 制作 OTA 包常见问题和注意事项 29

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 前言

1.1 概述

- 本文档提供 OTA 升级量产问题快速排查指导。
- 本文档提供的快速排查指南工程思路如下：常规问题场景 + 日志分析 + 解决思路。
- 本文档仅用于帮助开发人员快速排查和解决初级问题，如果按照文档中的指导仍无法解决问题，请联系全志原厂 FAE 进一步协助解决，并提供说明是否按照文档中的排查方案进行过必要的排查，提供排查过程的必要记录和数据。

1.2 读者对象

本文档主要适用于以下工程师。

- 对 OTA 升级技术 “零” 基础的研发或 FAE 工程人员。
- 对 OTA 升级初级基础的研发或 FAE 工程人员。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 OTA 问题分析手段

2.1 常见的 OTA 问题分类

1. OTA 打包问题。
2. update_engine 系统升级问题。
3. 升级后系统重启失败问题。

2.2 OTA 问题调试手段

1. 打开串口 log。

```
reboot uboot
printenv 查看cma的大小，如cma=8M。则设置cma为8M，并附带printk.devkmsg=on参数。
setenv cma 8M printk.devkmsg=on
setenv loglevel 8
saveenv
```

2. 在复现 OTA 升级后系统重启失败问题，接串口进行测试。
3. 对于 OTA 升级后系统启动失败问题，在系统起来的时候，立刻在串口中输入 logcat，抓取 logcat。
4. 排除 ddr 和 flash 物料的问题。
5. logcat 抓取 update_engine 的 log，或者抓取/data/misc/update_engine_log/目录下的 update_engine 的 log。抓取 logcat，然后搜索 update_engine。

```
logcat | grep "update_engine"
```

6. bootctl 命令。

```
通过bootctl可以查看当前的slot以及下一次重启要切换的slot等信息。
bootctl - command-line wrapper for the boot HAL.

Usage:
  bootctl COMMAND

Commands:
  hal-info              - Show info about boot_control HAL used.
  get-number-slots      - Prints number of slots.
  get-current-slot      - Prints currently running SLOT.
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
  mark-boot-successful          - Mark current slot as GOOD.
  get-active-boot-slot          - Prints the SLOT to load on next boot.
  set-active-boot-slot SLOT     - On next boot, load and execute SLOT.
  set-slot-as-unbootable SLOT   - Mark SLOT as invalid.
  is-slot-bootable SLOT         - Returns 0 only if SLOT is bootable.
  is-slot-marked-successful SLOT - Returns 0 only if SLOT is marked GOOD.
  get-suffix SLOT               - Prints suffix for SLOT.
  set-snapshot-merge-status STAT - Sets whether a snapshot-merge of any dynamic
                                   partition is in progress. Valid STAT values
                                   are: none, unknown, snapshotted, merging,
                                   or cancelled.
  get-snapshot-merge-status     - Prints the current snapshot-merge status.

例如执行get-current-slot，0代表A slot。
# bootctl get-current-slot
0
```

7. 获取当前 slot，OTA 升级之后，可以通过该命令查看是否切换到 B slot。

```
# getprop ro.boot.slot_suffix
_a
```

8. hexdump misc 分区

```
# busybox hexdump -C /dev/block/by-name/misc

misc分区用来记录slot的优先级，slot可重启的次数等数据。
可以通过该命令定位OTA升级，重启之后，A和B slot之间切换异常的问题。

00000000  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|  // bootloader_message数据结构
*
00000800  61 00 00 00 42 43 41 42  01 02 00 00 9f 00 7f 00  |a...BCAB........|  // slot_suffix bootloader_control
00000810  00 00 00 00 00 00 00 00  00 00 00 00 7c fd 96 02  |............|...|
00000820  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
00008000  02 b0 0a 74 56 00 00 00  00 00 00 00 00 00 00 00  |...tV...........|
00008010  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
01000000

slot_a_info        9f 00(1001 1111 0000 0000)
   priority        取1111，为15；
   tries_remaining  取001，为1；
   successful_boot  取1，为1；
slot_b_info        7f 00(0111 1111 0000 0000)
   priority        取1111，为15；
   tries_remaining  取111，为7；
   successful_boot  取0，为0；
```

2.3 OTA 问题 log 分析

1. 对于 OTA 打包问题，查看编译时报错的堆栈，定位对应的代码。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2. 对于 update_engine 系统升级问题，则直接抓取 logcat，然后搜索 update_engine。
3. 对于升级后系统重启失败问题，则需要打开串口 log，接串口进行测试，并在系统起来之后迅速抓取 logcat 信息，同时进一步排除 ddr 和 flash 物料的问题。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 OTA 打包问题

3.1 OTA 打包命令实现原理

OTA 打包命令源码：device/softwinner/common/vendorsetup.sh

OTA 打包命令实现原理。

1. 编译生成 target-files，make target-files-package otatools-package
2. 签名的 target-files-package

```
./build/tools/releasetools/sign_target_files_apks -d key_dir -o target_files sign_target_files
```

3. img_from_target_files

（1）img_from_target_files 从签了名之后的 signed_target_files.zip 中提取 img 文件。

（2）将签了名之后的 img 文件解压到 out 目录。

（3）pack 生成固件，就是签了名之后的固件了。

4. update_uboot

将 boot-resource.fex env.fex boot0_nand.fex boot0_sdcard.fex u-boot.fex toc1.fex toc0.fex 打包到 custom.img 中，并放到了 target-file.zip 的./IMAGES 目录中。

5. 生成完整包

```
./build/tools/releasetools/ota_from_target_files $k_arg $final_target_files $full_ota
```

6. 生成差分包

```
./build/tools/releasetools/ota_from_target_files $k_arg -i $old_target_files $final_target_files $inc_ota
```

注意

版权所有 © 珠海全志科技股份有限公司。保留一切权利

对于 Android 14 GRF 固件。

第一步生成 target-files 命令，make target-files-package otatools-package 替换为。

（1）生成 Android 13 的 target-files，先到 Android 13 sdk 目录中，执行 make target-files-package otatools-package 命令。

（2）生成 Android 14 的 target-files，然后到 Android 14 sdk 目录中，执行 make target-files-package otatools-package 命令。

（3）合并 Android 13 和 Android 14 的 target-files，执行 merge_target_files 命令。

3.2 OTA 包大于 2G，打包失败

3.2.1 问题描述

预置 3G 大小的 apk 到 system 分区中，ota 编译打包失败。

3.2.2 问题分析

查看 log 和代码，发现需要 ZIP64，可能是 32 位溢出了，导致编译打包失败。

```
Traceback (most recent call last):
  File "out/host/linux-x86/bin/img_from_target_files/internal/stdlib/runpy.py", line 196, in _run_module_as_main
  File "out/host/linux-x86/bin/img_from_target_files/internal/stdlib/runpy.py", line 86, in _run_code
  File "out/host/linux-x86/bin/img_from_target_files/__main__.py", line 12, in <module>
  File "out/host/linux-x86/bin/img_from_target_files/internal/stdlib/runpy.py", line 196, in _run_module_as_main
  File "out/host/linux-x86/bin/img_from_target_files/internal/stdlib/runpy.py", line 86, in _run_code
  File "out/host/linux-x86/bin/img_from_target_files/img_from_target_files.py", line 253, in <module>
  File "out/host/linux-x86/bin/img_from_target_files/img_from_target_files.py", line 245, in main
  File "out/host/linux-x86/bin/img_from_target_files/img_from_target_files.py", line 218, in ImgFromTargetFiles
  File "out/host/linux-x86/bin/img_from_target_files/img_from_target_files.py", line 174, in
      RebuildAndWriteSuperImages
  File "out/host/linux-x86/bin/img_from_target_files/internal/stdlib/zipfile.py", line 1308, in __exit__
  File "out/host/linux-x86/bin/img_from_target_files/internal/stdlib/zipfile.py", line 1833, in close
  File "out/host/linux-x86/bin/img_from_target_files/internal/stdlib/zipfile.py", line 1908, in _write_end_record
zipfile.LargeZipFile: Central directory offset would require ZIP64 extensions
```

3.2.3 问题解决方案

使用 ZIP64，将 allowZip64 设为 True。

```
diff --git a/tools/releasetools/img_from_target_files.py b/tools/releasetools/img_from_target_files.py
index 76da89c865..f8bdd81ce6 100755
--- a/tools/releasetools/img_from_target_files.py
+++ b/tools/releasetools/img_from_target_files.py
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
@@ -173,7 +173,7 @@ def RebuildAndWriteSuperImages(input_file, output_file):
  logger.info('Writing super.img to archive...')
  with zipfile.ZipFile(
      output_file, 'a', compression=zipfile.ZIP_DEFLATED,
-     allowZip64=not OPTIONS.sparse_userimages) as output_zip:
+     allowZip64=True) as output_zip:
  common.ZipWrite(output_zip, super_file, 'super.img')
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4 update_engine 系统升级问题

4.1 update_engine 系统升级流程

1. Update 应用通过 update_engine_client 接口，将 ota 包和升级参数发送给 update_engine。
2. update_engine 设置 A slot 为可启动，B slot 为不可启动。
3. update_engine 清除上一次更新。
4. update_engine 打印 install_plan。
5. update_engine 解压 ota 包，并将数据升级到 B slot 的分区。
6. update_engine 验证 B slot 的分区数据。
7. update_engine 升级 uboot 和 env 和 bootloader 分区。
8. update_engine 升级完成，重启系统。

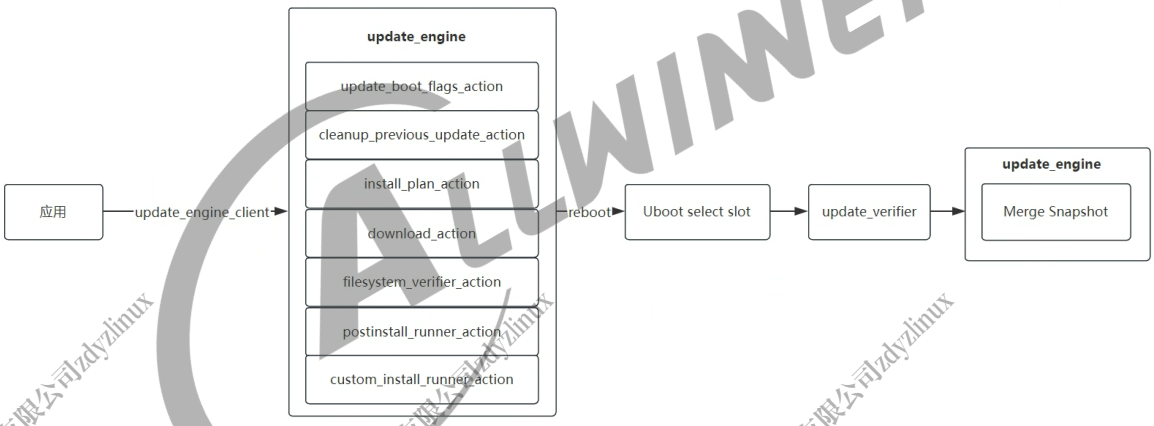

图 4-1: update_engine 系统升级流程

4.2 OTA 签名问题

4.2.1 问题描述

Update 应用显示在升级到 20% 的时候，OTA 升级失败。

4.2.2 问题分析

通过抓取 logcat，搜索 update_engine，可以发现 OTA 签名不一致，导致 OTA 升级失败。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
11-22 05:55:11.841  347  347 I update_engine: [INFO:download_action.cc(84)] Marking new slot as unbootable
11-22 05:55:11.853  347  347 I update_engine: [INFO:multi_range_http_fetcher.cc(45)] starting first transfer
11-22 05:55:11.858  347  347 I update_engine: [INFO:multi_range_http_fetcher.cc(74)] starting transfer of range
      2617+667157915
11-22 05:55:11.871 6283 6300 D AbUpdate: update progress: 2.8480514E-5
11-22 05:55:11.871  347  347 I update_engine: [INFO:delta_performer.cc(115)] Completed 0/? operations,
      16384/667157915 bytes downloaded (0%), overall progress 0%
11-22 05:55:11.877  347  347 I update_engine: [INFO:delta_performer.cc(346)] Manifest size in payload matches
      expected value from Omaha
11-22 05:55:11.882  347  347 I update_engine: [INFO:delta_performer.cc(1009)] Verifying using certificates: /system/etc/
      security/otacerts.zip
11-22 05:55:11.893  347  347 I update_engine: [INFO:payload_verifier.cc(102)] signature blob size = 267
11-22 05:55:11.898  347  347 I update_engine: [INFO:payload_verifier.cc(118)] Truncating the signature to its unpadded
      size: 256.
11-22 05:55:11.905  347  347 I update_engine: [INFO:payload_verifier.cc(191)] Failed to verify the signature with 1 keys.
11-22 05:55:11.910  347  347 E update_engine: [ERROR:payload_verifier.cc(137)] None of the 1 signatures is correct.
      Expected hash before padding:
11-22 05:55:11.915  347  347 I update_engine: [INFO:utils.cc(401)] Logging array of length: 32
11-22 05:55:11.921  347  347 I update_engine: [INFO:utils.cc(418)] 0x00000000 : 31 18 8a 02 24 d8 b9 20 97 3b 99 46 28 93
      6f 56
11-22 05:55:11.926  347  347 I update_engine: [INFO:utils.cc(418)] 0x00000010 : 3b f8 3e bc d4 9c 21 71 91 ea ad c3 e1 77
      db bd
11-22 05:55:11.932  347  347 E update_engine: [ERROR:payload_verifier.cc(140)] But found RSA decrypted hashes:
11-22 05:55:11.938  347  347 I update_engine: [INFO:utils.cc(401)] Logging array of length: 256
11-22 05:55:11.942  347  347 I update_engine: [INFO:utils.cc(418)] 0x00000000 : cc 9c b0 b3 73 c8 99 71 4e 50 83 28 84 c7
      11 41
11-22 05:55:11.949  347  347 I update_engine: [INFO:utils.cc(418)] 0x00000010 : e6 44 1b 9c 12 56 81 20 c4 b6 c4 bb e6 26
      0e e0
11-22 05:55:11.955  347  347 I update_engine: [INFO:utils.cc(418)] 0x00000020 : c0 78 11 b9 17 ac da e9 ff ce 72 06 91 44
      e8 70
11-22 05:55:11.960  347  347 I update_engine: [INFO:utils.cc(418)] 0x00000030 : 77 a6 f5 94 5b 27 43 7f b7 1a 4a e5 83 6d
      45 78
11-22 05:55:11.964  347  347 I update_engine: [INFO:utils.cc(418)] 0x00000040 : 7f bb b7 eb c9 4f 20 52 53 d2 0f 33 2f cb
      04 92
11-22 05:55:11.971  347  347 I update_engine: [INFO:utils.cc(418)] 0x00000050 : eb 06 7d f6 aa df 66 3d d5 33 b8 02 b1 03
      c1 a1
11-22 05:55:11.975  347  347 I update_engine: [INFO:utils.cc(418)] 0x00000060 : 87 8d eb b9 d5 ce d8 51 23 7b 01 eb 92 86
      f6 a7
11-22 05:55:11.980  347  347 I update_engine: [INFO:utils.cc(418)] 0x00000070 : 08 27 f4 4f a5 0d c8 85 75 be 73 fb 5f 56
      a2 56
11-22 05:55:11.985  347  347 I update_engine: [INFO:utils.cc(418)] 0x00000080 : 61 24 07 02 6f 2e 75 49 b1 5c b5 7f 2f d9 7
      d bc
11-22 05:55:11.989  347  347 I update_engine: [INFO:utils.cc(418)] 0x00000090 : 58 b2 b8 e6 b1 0a e8 6b 60 49 08 bc 86 4b
      b0 63
11-22 05:55:11.993  347  347 I update_engine: [INFO:utils.cc(418)] 0x000000a0 : 3b cf 54 b6 76 eb c5 ce 5f 70 88 0a af b1 1
      b bb
11-22 05:55:11.997  347  347 I update_engine: [INFO:utils.cc(418)] 0x000000b0 : 1d 5c e5 3c 08 6d a9 d1 0d d7 86 78 a5 18
      4d 20
11-22 05:55:12.002  347  347 I update_engine: [INFO:utils.cc(418)] 0x000000c0 : a4 24 a3 0c 85 f5 79 82 c2 f1 cc bb 0c cf 01
      ea
11-22 05:55:12.007  347  347 I update_engine: [INFO:utils.cc(418)] 0x000000d0 : 9e d3 ab cf 85 41 5f a1 7e 7f 37 f0 14 ae
      45 6d
11-22 05:55:12.012  347  347 I update_engine: [INFO:utils.cc(418)] 0x000000e0 : 42 2e 11 7b 1c 03 98 1b f7 cb 16 c9 87 d1
      5d 93
11-22 05:55:12.017  347  347 I update_engine: [INFO:utils.cc(418)] 0x000000f0 : d7 c3 63 51 03 be 13 a5 b4 fb 6a 50 2f 2e 8
      c 7d
11-22 05:55:12.021  347  347 E update_engine: [ERROR:payload_metadata.cc(214)] Manifest hash verification failed.
11-22 05:55:12.026  347  347 E update_engine: [ERROR:delta_performer.cc(374)] Mandatory metadata signature
      validation failed
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
11-22 05:55:12.033  347  347 E update_engine: [ERROR:download_action.cc(222)] Error ErrorCode::
      kDownloadMetadataSignatureMismatch (26) in DeltaPerformer's Write method when processing the received
      payload -- Terminating processing
11-22 05:55:12.038  347  347 I update_engine: [INFO:delta_performer.cc(217)] Discarding 48751 unused downloaded
      bytes
11-22 05:55:12.041  347  347 I update_engine: [INFO:multi_range_http_fetcher.cc(177)] Received transfer terminated.
11-22 05:55:12.045  347  347 I update_engine: [INFO:multi_range_http_fetcher.cc(129)] TransferEnded w/ code 200
11-22 05:55:12.050  347  347 I update_engine: [INFO:multi_range_http_fetcher.cc(131)] Terminating.
11-22 05:55:12.056  347  347 I update_engine: [INFO:action_processor.cc(116)] ActionProcessor: finished
      DownloadAction with code ErrorCode::kDownloadMetadataSignatureMismatch
```

4.2.3 问题原因

使用非 pack4dist 生成的固件去升级 OTA 包，固件是非签名的，OTA 包是签名的，导致固件和 OTA 包的签名不匹配，OTA 升级失败。

4.2.4 问题解决方案

1. 查看当前机器的固件是否是签名固件。fingerprint 末尾为 release-keys 则是签名固件，test-keys 则是非签名固件。

```
getprop | grep fingerprint

release-keys签名固件
# getprop |grep fingerprint
[ro.build.fingerprint]: [Allwinner/a523_pro_arm64/a523-pro:14/UP1A.231105.001.A1/eng.dailyb.20231121.043505:
      userdebug/release-keys]

test-keys非签名固件
# getprop | grep fingerprint
[ro.build.fingerprint]: [Allwinner/a523_pro_arm64/a523-pro:13/TQ2A.230405.003.B2/eng.20231117.112010:userdebug/
      test-keys]
```

如果是非签名固件，则需要先烧写 pack4dist 生成的签名固件，然后再升级对应的 OTA 包，才能升级成功。

注意

1. pack4dist 命令是判断 sdk 中 vendor/security/app_keys 目录是否存在，然后去生成签名的 OTA 包。
2. Android 11 及以上版本，vendor/security/app_keys 目录是默认存在的，所以 pack4dist 命令生成的 OTA 包都是默认带签名的。
3. 如果不想生成签名的 OTA 包，可以删掉 vendor/security/app_keys 目录。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.3 OTA 差分包版本不一致问题

4.3.1 问题描述

Update 应用显示在升级到 20% 的时候，OTA 升级失败。

4.3.2 问题分析

通过抓取 logcat，搜索 update_engine，可以发现 OTA 的 boot 分区 hash 校验不一致，导致 OTA 升级失败。

```
11-23 16:33:04.588  661  661 I update_engine: [INFO:delta_performer.cc(268)] PartitionInfo new custom sha256: 934
      A46525E1368DA09657D00999825DA2B65B76ACBB6DB53D063CBC563F031EC size: 3149824
11-23 16:33:04.600  661  661 I update_engine: [INFO:partition_writer_factory_android.cc(42)] Virtual AB Compression
      disabled, using Partition Writer for `boot`
11-23 16:33:04.602  661  661 I update_engine: [INFO:partition_writer.cc(175)] Opening /dev/block/by-name/boot_b
      partition without O_DSYNC
11-23 16:33:04.610  661  661 I update_engine: [INFO:partition_writer.cc(103)] Caching writes.
11-23 16:33:04.613  661  661 I update_engine: [INFO:partition_writer.cc(187)] Applying 62 operations to partition "boot"
11-23 16:33:04.641  661  661 I update_engine: [INFO:delta_performer.cc(580)] Starting to apply update payload
      operations
--------- beginning of kernel
11-23 16:33:05.811  661  661 E fec    : <661> error: parse_avb_image: failed to find avb hashtree descriptor
11-23 16:33:05.820  661  661 E fec    : <661> error: fec_open: failed to parse avb image.
11-23 16:33:05.614  661  661 E update_engine: [ERROR:verified_source_fd.cc(50)] Unable to open ECC source partition /
      dev/block/by-name/boot_a: Success (0)
11-23 16:33:05.617  661  661 E update_engine: [ERROR:partition_writer.cc(325)] The hash of the source data on disk for
      this operation doesn't match the expected value. This could mean that the delta update payload was targeted for
      another version, or that the source partition was modified after it was installed, for example, by mounting a
      filesystem.
11-23 16:33:05.619  661  661 E update_engine: [ERROR:partition_writer.cc(330)] Expected: sha256|hex = 62598
      EFB56A688AF9D9FFC31CCD90571F1BEE68A3D35ADECD4E694CC9E338493
11-23 16:33:05.625  661  661 E update_engine: [ERROR:partition_writer.cc(333)] Calculated: sha256|hex =
      CB0DA13AEBADC3CA3F43F0601432B48E355CF1C651CFBE800B326F4C16A8FB57
11-23 16:33:05.627  661  661 E update_engine: [ERROR:partition_writer.cc(344)] Operation source (offset:size) in blocks:
      3:14,7008:5,11420:1
11-23 16:33:05.629  661  661 E update_engine: [ERROR:partition_writer.cc(267)] source_fd != nullptr failed.
11-23 16:33:05.631  661  661 E update_engine: [ERROR:delta_performer.cc(944)] partition_writer_->
      PerformDiffOperation( operation, error, buffer_.data(), buffer_.size()) failed.
11-23 16:33:05.633  661  661 E update_engine: [ERROR:delta_performer.cc(195)] Failed to perform BROTLI_BSDIFF
      operation 50, which is the operation 50 in partition "boot"
11-23 16:33:05.634  661  661 E update_engine: [ERROR:download_action.cc(226)] Error ErrorCode::
      kDownloadStateInitializationError (20) in DeltaPerformer's Write method when processing the received payload --
      Terminating processing
11-23 16:33:06.070  661  661 I update_engine: [INFO:delta_performer.cc(214)] Discarding 185 unused downloaded bytes
11-23 16:33:06.150  661  661 I update_engine: [INFO:multi_range_http_fetcher.cc(177)] Received transfer terminated.
11-23 16:33:06.155  661  661 I update_engine: [INFO:multi_range_http_fetcher.cc(129)] TransferEnded w/ code 200
11-23 16:33:06.160  661  661 I update_engine: [INFO:multi_range_http_fetcher.cc(131)] Terminating.
11-23 16:33:06.164  661  661 I update_engine: [INFO:action_processor.cc(116)] ActionProcessor: finished
      DownloadAction with code ErrorCode::kDownloadStateInitializationError
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.3.3 问题解决方案

解压 OTA 包，打开 metadata 文件，确认 pre-build 字段的 fingerprint 是否和当前机器的 fingerprint 一致。

如果不一致，则需要找到机器固件对应的 target_file，再生成一个差分包，才能升级成功。

```
META-INF\com\android\metadata文件的fingerprint。
pre-build=Allwinner/a523_pro_arm64/a523-pro:14/UP1A.231105.001.A1/eng.dailyb.20231122.044732:userdebug/
        release-keys

查看当前机器的fingerprint。
/ # getprop ro.build.fingerprint
Allwinner/a523_pro_arm64/a523-pro:14/UP1A.231105.001.A1/eng.dailyb.20231121.043505:userdebug/release-keys
```

4.4 OTA 包损坏问题

4.4.1 问题描述

Update 应用显示在升级到 20% 的时候，OTA 升级失败。

4.4.2 问题分析

通过抓取 logcat，搜索 update_engine，可以发现 OTA 的 boot 分区 hash 校验不一致，导致 OTA 升级失败。

```
11-23 16:33:04.588  661  661 I update_engine: [INFO:delta_performer.cc(268)] PartitionInfo new custom sha256: 934
        A46525E1368DA09657D00999825DA2B65B76ACBB6DB53D063CBC563F031EC size: 3149824
11-23 16:33:04.600  661  661 I update_engine: [INFO:partition_writer_factory_android.cc(42)] Virtual AB Compression
        disabled, using Partition Writer for `boot`
11-23 16:33:04.602  661  661 I update_engine: [INFO:partition_writer.cc(175)] Opening /dev/block/by-name/boot_b
        partition without O_DSYNC
11-23 16:33:04.610  661  661 I update_engine: [INFO:partition_writer.cc(103)] Caching writes.
11-23 16:33:04.613  661  661 I update_engine: [INFO:partition_writer.cc(187)] Applying 62 operations to partition "boot"
11-23 16:33:04.641  661  661 I update_engine: [INFO:delta_performer.cc(580)] Starting to apply update payload
        operations
--------- beginning of kernel
11-23 16:33:05.811  661  661 E fec    : <661> error: parse_avb_image: failed to find avb hashtree descriptor
11-23 16:33:05.820  661  661 E fec    : <661> error: fec_open: failed to parse avb image.
11-23 16:33:05.614  661  661 E update_engine: [ERROR:verified_source_fd.cc(50)] Unable to open ECC source partition /
        dev/block/by-name/boot_a: Success (0)
11-23 16:33:05.617  661  661 E update_engine: [ERROR:partition_writer.cc(325)] The hash of the source data on disk for
        this operation doesn't match the expected value. This could mean that the delta update payload was targeted for
        another version, or that the source partition was modified after it was installed, for example, by mounting a
        filesystem.
11-23 16:33:05.619  661  661 E update_engine: [ERROR:partition_writer.cc(330)] Expected:   sha256|hex = 62598
        EFB56A688AF9D9FFC31CCD90571F1BEE68A3D35ADECD4E694CC9E338493
11-23 16:33:05.625  661  661 E update_engine: [ERROR:partition_writer.cc(333)] Calculated: sha256|hex =
        CB0DA13AEBADC3CA3F43F0601432B48E355CF1C651CFBE800B326F4C16A8FB57
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
11-23 16:33:05.627  661  661 E update_engine: [ERROR:partition_writer.cc(344)] Operation source (offset:size) in blocks:
      3:14,7008:5,11420:1
11-23 16:33:05.629  661  661 E update_engine: [ERROR:partition_writer.cc(267)] source_fd != nullptr failed.
11-23 16:33:05.631  661  661 E update_engine: [ERROR:delta_performer.cc(944)] partition_writer_->
      PerformDiffOperation( operation, error, buffer_.data(), buffer_.size()) failed.
11-23 16:33:05.633  661  661 E update_engine: [ERROR:delta_performer.cc(195)] Failed to perform BROTLI_BSDIFF
      operation 50, which is the operation 50 in partition "boot"
11-23 16:33:05.634  661  661 E update_engine: [ERROR:download_action.cc(226)] Error ErrorCode::
      kDownloadStateInitializationError (20) in DeltaPerformer's Write method when processing the received payload --
      Terminating processing
11-23 16:33:06.070  661  661 I update_engine: [INFO:delta_performer.cc(214)] Discarding 185 unused downloaded bytes
11-23 16:33:06.150  661  661 I update_engine: [INFO:multi_range_http_fetcher.cc(177)] Received transfer terminated.
11-23 16:33:06.155  661  661 I update_engine: [INFO:multi_range_http_fetcher.cc(129)] TransferEnded w/ code 200
11-23 16:33:06.160  661  661 I update_engine: [INFO:multi_range_http_fetcher.cc(131)] Terminating.
11-23 16:33:06.164  661  661 I update_engine: [INFO:action_processor.cc(116)] ActionProcessor: finished
      DownloadAction with code ErrorCode::kDownloadStateInitializationError
```

4.4.3 问题解决方案

根据上面 OTA 差分包版本不一致的步骤排除之后，需要确认下 OTA 包是否损坏了。

```
md5sum计算小机端的ota包的hash值
# md5sum ota.zip
c15dfccd2beefa5131604066988faf5d  ota.zip

md5sum计算服务器端的ota包的hash值
# md5sum ota.zip
7ec2ff13c1dd63d30ed64191e8297a27  ota.zip

如果不一致，则可能是ota包拷贝到小机端的时候，损坏了。
需要确认下是否是u盘拷贝的时候，没弹出。或者是小机端ddr和flash不稳定导致的。
```

4.5 remount 之后 OTA 升级失败问题

4.5.1 问题描述

Update 应用显示在升级到 20% 的时候，OTA 升级失败。

4.5.2 问题分析

通过抓取 logcat，搜索 update_engine，可以发现 OTA 检测到机器 remount 了，log 中 enable-verity 字段，导致 OTA 升级失败。

```
11-23 16:47:21.803  681  681 I update_engine: [INFO:delta_performer.cc(833)] Preparing partitions for new update. last
      hash = , new hash = 67Lqi4gI6ThYWpTm4PA3rCDggzP+XhdPXBYiWV0BdbI=27l1rjQ0UJwFkyVGltmnVjFRd/
      BgFO2LsDqxp7RMBJg=
11-23 16:47:21.913  681  681 I update_engine: [INFO:delta_performer.cc(1387)] Resetting recorded hash for prepared
      partitions.
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
11-23 16:47:21.931  681  681 W update_engine: [WARNING:dynamic_partition_control_android.cc(461)] overlayfs
      overrides are active and can interfere with our resources.
11-23 16:47:21.931  681  681 W update_engine: run adb enable-verity to deactivate if required and try again.
11-23 16:47:21.934  681  681 I update_engine: [INFO:snapshot.cpp(3835)] EnsureMetadataMounted does nothing in
      Android mode.
11-23 16:47:21.938  681  681 I update_engine: [INFO:dynamic_partition_control_android.cc(785)] Erasing AVB footer of
      system_other partition before update.
11-23 16:47:21.943  681  681 E update_engine: [ERROR:fs_mgr_fstab.cpp(705)] [libfs_mgr] ReadFstabFromFile(): failed
      to read file: '/system/etc/fstab.postinstall': No such file or directory
11-23 16:47:21.946  681  681 W update_engine: [WARNING:dynamic_partition_control_android.cc(659)] Cannot read
      fstab from /system/etc/fstab.postinstall: No such file or directory (2)
11-23 16:47:21.948  681  681 I update_engine: [INFO:dynamic_partition_control_android.cc(686)] AVB is not enabled on
      system_other. Skip erasing.
11-23 16:47:21.959  681  681 I update_engine: [INFO:dynamic_partition_control_android.cc(351)] Loaded metadata
      from slot A in /dev/block/by-name/super
11-23 16:47:21.986  681  681 E update_engine: [ERROR:snapshot.cpp(3153)] Cannot create update snapshots with
      overlayfs setup. Run `adb enable-verity`, reboot, then try again.
11-23 16:47:21.993  681  681 E update_engine: [ERROR:dynamic_partition_control_android.cc(975)] Cannot create
      update snapshots: Error
11-23 16:47:21.997  681  681 E update_engine: [ERROR:dynamic_partition_control_android.cc(520)]
      PrepareSnapshotPartitionsForUpdate failed in Android mode
11-23 16:47:22.000  681  681 E update_engine: [ERROR:delta_performer.cc(844)] Unable to initialize partition metadata
      for slot B
11-23 16:47:22.002  681  681 E update_engine: [ERROR:download_action.cc(226)] Error ErrorCode::
      kInstallDeviceOpenError (7) in DeltaPerformer's Write method when processing the received payload -- Terminating
       processing
11-23 16:47:22.304  681  681 I update_engine: [INFO:multi_range_http_fetcher.cc(177)] Received transfer terminated.
11-23 16:47:22.317  681  681 I update_engine: [INFO:multi_range_http_fetcher.cc(129)] TransferEnded w/ code 200
11-23 16:47:22.323  681  681 I update_engine: [INFO:multi_range_http_fetcher.cc(131)] Terminating.
11-23 16:47:22.330  681  681 I update_engine: [INFO:action_processor.cc(116)] ActionProcessor: finished
      DownloadAction with code ErrorCode::kInstallDeviceOpenError
```

4.5.3 问题解决方案

执行 adb enable-verity 命令，或者机器恢复出厂设置，或者重新分区格式化烧写固件，然后进行 OTA 升级。

4.6 dragonface 修改固件之后，ota 升级失败

4.6.1 问题描述

使用 dragonface 固件修改工具修改开机动画之后，进行 ota 升级，ota 升级失败。

4.6.2 问题分析

通过抓取 logcat，搜索 update_engine，可以发现 update_engine 升级过程中，vendor_boot 分区校验失败，导致 OTA 升级失败。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
09-11 05:06:57.826  2627  2627 I update_engine: [INFO:dynamic_partition_control_android.cc(319)] Loaded metadata
      from slot A in /dev/block/by-name/super
09-11 05:06:57.835  2627  2627 I update_engine: [INFO:dynamic_partition_control_android.cc(957)] vendor_a is mapped
      on device mapper: /dev/block/dm-1
09-11 05:07:00.286  2969  3775 E WifiVendorHal: getWifiLinkLayerStats_1_3_Internal(l.973) failed {.code =
      ERROR_UNKNOWN, .description = unknown error}
09-11 05:07:01.594  2627  2627 I update_engine: [INFO:dynamic_partition_control_android.cc(319)] Loaded metadata
      from slot A in /dev/block/by-name/super
09-11 05:07:01.596  2627  2627 I update_engine: [INFO:dynamic_partition_control_android.cc(938)] boot_a is not in
      super partition metadata.
09-11 05:07:01.600  2627  2627 I update_engine: [INFO:dynamic_partition_control_android.cc(319)] Loaded metadata
      from slot A in /dev/block/by-name/super
09-11 05:07:03.175  2627  2627 I update_engine: [INFO:dynamic_partition_control_android.cc(319)] Loaded metadata
      from slot A in /dev/block/by-name/super
09-11 05:07:03.185  2627  2627 I update_engine: [INFO:dynamic_partition_control_android.cc(938)] vendor_boot_a is
      not in super partition metadata.
09-11 05:07:03.192  2627  2627 I update_engine: [INFO:dynamic_partition_control_android.cc(319)] Loaded metadata
      from slot A in /dev/block/by-name/super
09-11 05:07:03.212  2627  2627 E update_engine: [ERROR:delta_performer.cc(1161)] The hash of the source data on disk
      for this operation doesn't match the expected value. This could mean that the delta update payload was targeted
      for another version, or that the source partition was modified after it was installed, for example, by mounting a
      filesystem.
09-11 05:07:03.219  2627  2627 E update_engine: [ERROR:delta_performer.cc(1166)] Expected:   sha256|hex = 1059629
      D738FD8CC3FCF1E5C29C402EF6008E65CC61D4EF8A34DC9FF123F28D1
09-11 05:07:03.224  2627  2627 E update_engine: [ERROR:delta_performer.cc(1169)] Calculated: sha256|hex = 0
      FE8BBE02DA317DDC53A2DD2A9AB404B5DA3D6AC8195BB1CD3DA455BA3A93992
09-11 05:07:03.228  2627  2627 E update_engine: [ERROR:delta_performer.cc(1180)] Operation source (offset:size) in
      blocks: 0:31,8191:1
```

4.6.3 问题原因

dragonface 固件修改工具重新打包 vendor_boot 分区导致的。

4.6.4 问题解决方案

使用 dragonface 固件修改工具修改模型的时候，注意不要勾选 1 板卡配置，板卡配置会改动 vendor_boot 分区。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5 升级后系统重启失败问题

5.1 升级后系统重启流程

1. uboot 选择从 B slot 启动系统。

升级完成后重启，uboot根据misc分区的数据切换到对应的slot。具体判断逻辑如下，

```
sunxi_replace_android_ab_system
  -> ab_select_slot_from_partname
    -> ab_select_slot
      -> 寻找slot:
        -> tries_remaining大于0表示可以启动
        -> ab_compare_slots：如果_a和_b的tries_remaining都大于0，则选择一下:
          -> priority的值，谁高用谁
          -> successful_boot是否一致，不一致则看谁设了1
          -> tries_remaining的值，谁高用谁
          -> 以上都不行，则用_a
      -> 如果找到的slot其successful_boot值为0，则表示该slot在OTA中，将其tries_remaining值减1。如果后面OTA升级一直失败重启，直到tries_remaining减到0，则该slot为不可启动。这时候，根据上面的逻辑，就会回退到升级前的分区进行启动了。
```

2. update_verifier 标记 B slot 分区为可启动。
3. 系统 boot_completed 之后，update_engine 合并 B slot 分区数据到 super 分区中。

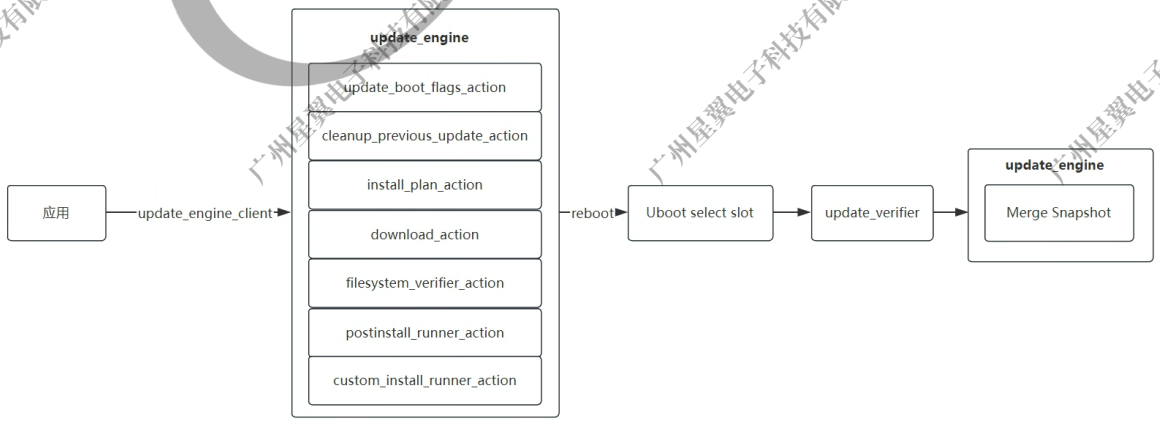

图 5-1: 升级后系统重启流程

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5.2 OTA 升级后，多次断电启动，版本回退至 OTA 前版本

5.2.1 问题描述

OTA 升级后，多次断电启动，版本回退至 OTA 前版本。

5.2.2 问题分析

从串口 log 中，可以看到 update_verifier 的异常。isSlotMarkedSuccessful 为-1，就不会调用 module->markBootSuccessful 设置 B slot 为成功启动，导致多次重启后，版本回退到 OTA 前版本。

```
11-23 16:30:08.166177[   27.840295] update_verifier: Booting slot 0: isSlotMarkedSuccessful=-1
```

5.2.3 问题解决方案

bootctl hal isSlotMarkedSuccessful 接口不应该返回-1。

```
hardware/aw/bootctrl目录
diff --git a/legacy_boot_control.cpp b/legacy_boot_control.cpp
index c42fc69..0db23f7 100644
--- a/legacy_boot_control.cpp
+++ b/legacy_boot_control.cpp
@@ -69,7 +69,7 @@ int BootControl_isSlotBootable(struct boot_control_module* module, unsigned int

 int BootControl_isSlotMarkedSuccessful(struct boot_control_module* module, unsigned int slot) {
  auto& impl = reinterpret_cast<boot_control_private_t*>(module)->impl;
-  return impl.IsSlotMarkedSuccessful(slot) ? 0 : -1;
+  return impl.IsSlotMarkedSuccessful(slot);
 }

 const char* BootControl_getSuffix(boot_control_module_t* module, unsigned int slot) {
```

5.3 ota 包大于 2G 的时候 OTA 升级失败问题

5.3.1 问题描述

使用大于 2G 的 ota 包进行 OTA 升级，update_engine 升级成功，重启后系统启动不到 launcher 主界面。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5.3.2 问题分析

从串口 log 看，内核 dm-user 出现报错，是因为 snapuserd 进程被杀导致的。

```
[  91.426211][  T26] I/O error: sector 4724752: no user-space daemon for dm-user/product_b target
[  91.537115][  T26] I/O error: sector 13056: no user-space daemon for dm-user/vendor_dlkm_b target
[  91.552086][  T26] I/O error: sector 13056: no user-space daemon for dm-user/vendor_dlkm_b target
[  91.591502][  T61] I/O error: sector 12864: no user-space daemon for dm-user/vendor_dlkm_b target
```

5.3.3 问题原因

通过添加 log，发现在特定地方 snapuserd 进程出现线程终止，经过分析均为读写文件，并且偏移值超过 2G。

32 bit 的 android off_t 定义为 long，所以在 32bit 的取值范围为-2,147,483,648 到 2,147,483,647，所以导致 io_uring，pread，pwrite 的接口访问超过 2G 的偏移会变成负值，导致 snapuserd 线程被 kill 掉。

5.3.4 问题解决方案

使用 io_uring_prep_rw 和 pread64 和 pwrite64 代替原来的接口，防止溢出。

```
diff --git a/fs_mgr/fs_mgr.cpp b/fs_mgr/fs_mgr.cpp
index f170bb912..96f85bb78 100644
--- a/fs_mgr/fs_mgr.cpp
+++ b/fs_mgr/fs_mgr.cpp
@@ -342,7 +342,7 @@ static bool read_ext4_superblock(const std::string& blk_device, struct ext4_supe
       return false;
    }

-   if (TEMP_FAILURE_RETRY(pread(fd, sb, sizeof(*sb), 1024)) != sizeof(*sb)) {
+   if (TEMP_FAILURE_RETRY(pread64(fd, sb, sizeof(*sb), 1024)) != sizeof(*sb)) {
       PERROR << "Can't read '" << blk_device << "' superblock";
       return false;
    }
@@ -367,7 +367,7 @@ bool fs_mgr_is_ext4(const std::string& blk_device) {
    android::base::unique_fd fd(TEMP_FAILURE_RETRY(open(blk_device.c_str(), O_RDONLY | O_CLOEXEC)));
    if (fd < 0) return false;
    ext4_super_block sb;
-   if (TEMP_FAILURE_RETRY(pread(fd, &sb, sizeof(sb), 1024)) != sizeof(sb)) return false;
+   if (TEMP_FAILURE_RETRY(pread64(fd, &sb, sizeof(sb), 1024)) != sizeof(sb)) return false;
    if (!is_ext4_superblock_valid(&sb)) return false;
    return true;
 }
@@ -642,11 +642,11 @@ static bool read_f2fs_superblock(const std::string& blk_device, int* fs_stat) {
       return false;
    }

-   if (TEMP_FAILURE_RETRY(pread(fd, &sb1, sizeof(sb1), F2FS_SUPER_OFFSET)) != sizeof(sb1)) {
+   if (TEMP_FAILURE_RETRY(pread64(fd, &sb1, sizeof(sb1), F2FS_SUPER_OFFSET)) != sizeof(sb1)) {
       PERROR << "Can't read '" << blk_device << "' superblock1";
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    return false;
  }
-   if (TEMP_FAILURE_RETRY(pread(fd, &sb2, sizeof(sb2), F2FS_BLKSIZE + F2FS_SUPER_OFFSET)) !=
+   if (TEMP_FAILURE_RETRY(pread64(fd, &sb2, sizeof(sb2), F2FS_BLKSIZE + F2FS_SUPER_OFFSET)) !=
      sizeof(sb2)) {
      PERROR << "Can't read '" << blk_device << "' superblock2";
      return false;
@@ -666,11 +666,11 @@ bool fs_mgr_is_f2fs(const std::string& blk_device) {
   android::base::unique_fd fd(TEMP_FAILURE_RETRY(open(blk_device.c_str(), O_RDONLY | O_CLOEXEC)));
   if (fd < 0) return false;
   __le32 sb;
-   if (TEMP_FAILURE_RETRY(pread(fd, &sb, sizeof(sb), F2FS_SUPER_OFFSET)) != sizeof(sb)) {
+   if (TEMP_FAILURE_RETRY(pread64(fd, &sb, sizeof(sb), F2FS_SUPER_OFFSET)) != sizeof(sb)) {
      return false;
   }
   if (sb == cpu_to_le32(F2FS_SUPER_MAGIC)) return true;
-   if (TEMP_FAILURE_RETRY(pread(fd, &sb, sizeof(sb), F2FS_BLKSIZE + F2FS_SUPER_OFFSET)) !=
+   if (TEMP_FAILURE_RETRY(pread64(fd, &sb, sizeof(sb), F2FS_BLKSIZE + F2FS_SUPER_OFFSET)) !=
      sizeof(sb)) {
      return false;
   }
diff --git a/fs_mgr/libsnapshot/cow_reader.cpp b/fs_mgr/libsnapshot/cow_reader.cpp
index 75a58a6fc..6573cb963 100644
--- a/fs_mgr/libsnapshot/cow_reader.cpp
+++ b/fs_mgr/libsnapshot/cow_reader.cpp
@@ -71,14 +71,14 @@ bool CowReader::InitForMerge(android::base::unique_fd&& fd) {
   owned_fd_ = std::move(fd);
   fd_ = owned_fd_.get();

-   auto pos = lseek(fd_.get(), 0, SEEK_END);
+   auto pos = lseek64(fd_.get(), 0, SEEK_END);
   if (pos < 0) {
      PLOG(ERROR) << "lseek end failed";
      return false;
   }
   fd_size_ = pos;

-   if (lseek(fd_.get(), 0, SEEK_SET) < 0) {
+   if (lseek64(fd_.get(), 0, SEEK_SET) < 0) {
      PLOG(ERROR) << "lseek header failed";
      return false;
   }
@@ -98,14 +98,14 @@ bool CowReader::Parse(android::base::unique_fd&& fd, std::optional<uint64_t> lab
 bool CowReader::Parse(android::base::borrowed_fd fd, std::optional<uint64_t> label) {
   fd_ = fd;

-   auto pos = lseek(fd_.get(), 0, SEEK_END);
+   auto pos = lseek64(fd_.get(), 0, SEEK_END);
   if (pos < 0) {
      PLOG(ERROR) << "lseek end failed";
      return false;
   }
   fd_size_ = pos;

-   if (lseek(fd_.get(), 0, SEEK_SET) < 0) {
+   if (lseek64(fd_.get(), 0, SEEK_SET) < 0) {
      PLOG(ERROR) << "lseek header failed";
      return false;
   }
@@ -168,13 +168,13 @@ bool CowReader::ParseOps(std::optional<uint64_t> label) {
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    if (header_.major_version >= 2 && (header_.buffer_size > 0)) {
      LOG(DEBUG) << " Scratch space found of size: " << header_.buffer_size;
      size_t init_offset = header_.header_size + header_.buffer_size;
-     pos = lseek(fd_.get(), init_offset, SEEK_SET);
+      pos = lseek64(fd_.get(), init_offset, SEEK_SET);
      if (pos != init_offset) {
        PLOG(ERROR) << "lseek ops failed";
        return false;
      }
    } else {
-     pos = lseek(fd_.get(), header_.header_size, SEEK_SET);
+      pos = lseek64(fd_.get(), header_.header_size, SEEK_SET);
      if (pos != header_.header_size) {
        PLOG(ERROR) << "lseek ops failed";
        return false;
@@ -229,7 +229,7 @@ bool CowReader::ParseOps(std::optional<uint64_t> label) {
          footer_.emplace();
          CowFooter* footer = &footer_.value();
          memcpy(&footer_->op, &current_op, sizeof(footer->op));
-         off_t offs = lseek(fd_.get(), pos, SEEK_SET);
+          off_t offs = lseek64(fd_.get(), pos, SEEK_SET);
          if (offs < 0 || pos != static_cast<uint64_t>(offs)) {
            PLOG(ERROR) << "lseek next op failed " << offs;
            return false;
@@ -249,7 +249,7 @@ bool CowReader::ParseOps(std::optional<uint64_t> label) {
      }

      // Position for next cluster read
-     off_t offs = lseek(fd_.get(), pos, SEEK_SET);
+      off_t offs = lseek64(fd_.get(), pos, SEEK_SET);
      if (offs < 0 || pos != static_cast<uint64_t>(offs)) {
        PLOG(ERROR) << "lseek next op failed " << offs;
        return false;
@@ -712,7 +712,7 @@ bool CowReader::GetRawBytes(uint64_t offset, void* buffer, size_t len, size_t* r
      LOG(ERROR) << "invalid data offset: " << offset << ", " << len << " bytes";
      return false;
    }
-   if (lseek(fd_.get(), offset, SEEK_SET) < 0) {
+   if (lseek64(fd_.get(), offset, SEEK_SET) < 0) {
      PLOG(ERROR) << "lseek to read raw bytes failed";
      return false;
    }
diff --git a/fs_mgr/libsnapshot/cow_writer.cpp b/fs_mgr/libsnapshot/cow_writer.cpp
index 5ce1d3bb2..f6744ed74 100644
--- a/fs_mgr/libsnapshot/cow_writer.cpp
+++ b/fs_mgr/libsnapshot/cow_writer.cpp
@@ -210,7 +210,7 @@ bool CowWriter::OpenForWrite() {
      return false;
    }

-   if (lseek(fd_.get(), 0, SEEK_SET) < 0) {
+   if (lseek64(fd_.get(), 0, SEEK_SET) < 0) {
      PLOG(ERROR) << "lseek failed";
      return false;
    }
@@ -240,7 +240,7 @@ bool CowWriter::OpenForWrite() {
      return false;
    }

-   if (lseek(fd_.get(), sizeof(header_) + header_.buffer_size, SEEK_SET) < 0) {
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
+   if (lseek64(fd_.get(), sizeof(header_) + header_.buffer_size, SEEK_SET) < 0) {
      PLOG(ERROR) << "lseek failed";
      return false;
    }
@@ -275,7 +275,7 @@ bool CowWriter::OpenForAppend(uint64_t label) {
    // Free reader so we own the descriptor position again.
    reader = nullptr;

-   if (lseek(fd_.get(), next_op_pos_, SEEK_SET) < 0) {
+   if (lseek64(fd_.get(), next_op_pos_, SEEK_SET) < 0) {
      PLOG(ERROR) << "lseek failed";
      return false;
    }
@@ -465,7 +465,7 @@ bool CowWriter::Finalize() {
      auto unused_cluster_space = cluster_size_ - current_cluster_size_;
      std::string clr;
      clr.resize(unused_cluster_space, '\0');
-     if (lseek(fd_.get(), next_op_pos_, SEEK_SET) < 0) {
+     if (lseek64(fd_.get(), next_op_pos_, SEEK_SET) < 0) {
        PLOG(ERROR) << "Failed to seek to footer position.";
        return false;
      }
@@ -483,7 +483,7 @@ bool CowWriter::Finalize() {
    }

    footer_.op.ops_size = ops_.size();
-   if (lseek(fd_.get(), next_op_pos_, SEEK_SET) < 0) {
+   if (lseek64(fd_.get(), next_op_pos_, SEEK_SET) < 0) {
      PLOG(ERROR) << "Failed to seek to footer position.";
      return false;
    }
@@ -501,7 +501,7 @@ bool CowWriter::Finalize() {

    // Remove excess data, if we're in append mode and threw away more data
    // than we wrote before.
-   off_t offs = lseek(fd_.get(), 0, SEEK_CUR);
+   off_t offs = lseek64(fd_.get(), 0, SEEK_CUR);
    if (offs < 0) {
      PLOG(ERROR) << "Failed to lseek to find current position";
      return false;
@@ -531,7 +531,7 @@ uint64_t CowWriter::GetCowSize() {
 }

 bool CowWriter::GetDataPos(uint64_t* pos) {
-   off_t offs = lseek(fd_.get(), 0, SEEK_CUR);
+   off_t offs = lseek64(fd_.get(), 0, SEEK_CUR);
    if (offs < 0) {
      PLOG(ERROR) << "lseek failed";
      return false;
@@ -541,7 +541,7 @@ bool CowWriter::GetDataPos(uint64_t* pos) {
 }

 bool CowWriter::WriteOperation(const CowOperation& op, const void* data, size_t size) {
-   if (lseek(fd_.get(), next_op_pos_, SEEK_SET) < 0) {
+   if (lseek64(fd_.get(), next_op_pos_, SEEK_SET) < 0) {
      PLOG(ERROR) << "lseek failed for writing operation.";
      return false;
    }
@@ -572,7 +572,7 @@ void CowWriter::AddOperation(const CowOperation& op) {
 }
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
 bool CowWriter::WriteRawData(const void* data, size_t size) {
-   if (lseek(fd_.get(), next_data_pos_, SEEK_SET) < 0) {
+   if (lseek64(fd_.get(), next_data_pos_, SEEK_SET) < 0) {
      PLOG(ERROR) << "lseek failed for writing data.";
      return false;
    }
@@ -598,7 +598,7 @@ bool CowWriter::Truncate(off_t length) {
    if (is_dev_null_ || is_block_device_) {
      return true;
    }
-   if (ftruncate(fd_.get(), length) < 0) {
+   if (ftruncate64(fd_.get(), length) < 0) {
      PLOG(ERROR) << "Failed to truncate.";
      return false;
    }
diff --git a/fs_mgr/libsnapshot/estimate_cow_from_nonab_ota.cpp b/fs_mgr/libsnapshot/
        estimate_cow_from_nonab_ota.cpp
index 45833e121..61d8253f3 100644
--- a/fs_mgr/libsnapshot/estimate_cow_from_nonab_ota.cpp
+++ b/fs_mgr/libsnapshot/estimate_cow_from_nonab_ota.cpp
@@ -138,7 +138,7 @@ unique_fd TargetFilesPackage::OpenFile(const std::string& path) {
      LOG(ERROR) << "could not extract " << path << " from " << path_;
      return {};
    }
-   if (lseek(temp.fd, 0, SEEK_SET) < 0) {
+   if (lseek64(temp.fd, 0, SEEK_SET) < 0) {
      PLOG(ERROR) << "lseek failed";
      return {};
    }
@@ -167,7 +167,7 @@ unique_fd TargetFilesPackage::OpenImage(const std::string& path) {
      LOG(ERROR) << "sparse_file_write failed";
      return {};
    }
-   if (lseek(temp.fd, 0, SEEK_SET) < 0) {
+   if (lseek64(temp.fd, 0, SEEK_SET) < 0) {
      PLOG(ERROR) << "lseek failed";
      return {};
    }
diff --git a/fs_mgr/libsnapshot/make_cow_from_ab_ota.cpp b/fs_mgr/libsnapshot/make_cow_from_ab_ota.cpp
index 6a5754de1..8cc271ea9 100644
--- a/fs_mgr/libsnapshot/make_cow_from_ab_ota.cpp
+++ b/fs_mgr/libsnapshot/make_cow_from_ab_ota.cpp
@@ -340,7 +340,7 @@ bool PayloadConverter::ProcessDiff(const InstallOperation& op) {
    // Read source bytes.
    for (const auto& extent : op.src_extents()) {
      uint64_t offset = extent.start_block() * kBlockSize;
-     if (lseek(source_image.get(), offset, SEEK_SET) < 0) {
+     if (lseek64(source_image.get(), offset, SEEK_SET) < 0) {
        PLOG(ERROR) << "lseek source image failed";
        return false;
      }
@@ -357,7 +357,7 @@ bool PayloadConverter::ProcessDiff(const InstallOperation& op) {

    // Read patch bytes.
    auto patch = std::make_unique<uint8_t[]>(op.data_length());
-   if (lseek(in_fd_.get(), payload_offset_ + op.data_offset(), SEEK_SET) < 0) {
+   if (lseek64(in_fd_.get(), payload_offset_ + op.data_offset(), SEEK_SET) < 0) {
      PLOG(ERROR) << "lseek payload failed";
      return false;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
   }
@@ -436,7 +436,7 @@ borrowed_fd PayloadConverter::OpenSourceImage() {
         LOG(ERROR) << "could not extract " << local_path;
         return -1;
       }
-      if (lseek(unzip_fd.get(), 0, SEEK_SET) < 0) {
+      if (lseek64(unzip_fd.get(), 0, SEEK_SET) < 0) {
         PLOG(ERROR) << "lseek failed";
         return -1;
       }
@@ -523,7 +523,7 @@ bool PayloadConverter::ProcessReplace(const InstallOperation& op) {
   auto buffer_size = op.data_length();
   auto buffer = std::make_unique<char[]>(buffer_size);
   uint64_t offs = payload_offset_ + op.data_offset();
-  if (lseek(in_fd_.get(), offs, SEEK_SET) < 0) {
+  if (lseek64(in_fd_.get(), offs, SEEK_SET) < 0) {
     PLOG(ERROR) << "lseek " << offs << " failed";
     return false;
   }
@@ -658,7 +658,7 @@ bool PayloadConverter::OpenPayload() {
   }

   // Skip past manifest signature.
-  auto offs = lseek(in_fd_, manifest_signature_size, SEEK_CUR);
+  auto offs = lseek64(in_fd_, manifest_signature_size, SEEK_CUR);
   if (offs < 0) {
     PLOG(ERROR) << "lseek failed";
     return false;
diff --git a/fs_mgr/libsnapshot/snapshot_reader.cpp b/fs_mgr/libsnapshot/snapshot_reader.cpp
index 6546c2a5c..ecb6daf71 100644
--- a/fs_mgr/libsnapshot/snapshot_reader.cpp
+++ b/fs_mgr/libsnapshot/snapshot_reader.cpp
@@ -53,7 +53,7 @@ ssize_t ReadFdFileDescriptor::Read(void* buf, size_t count) {
 }

 off64_t ReadFdFileDescriptor::Seek(off64_t offset, int whence) {
-  return lseek(fd_.get(), offset, whence);
+  return lseek64(fd_.get(), offset, whence);
 }

 uint64_t ReadFdFileDescriptor::BlockDevSize() {
diff --git a/fs_mgr/libsnapshot/snapshot_writer.cpp b/fs_mgr/libsnapshot/snapshot_writer.cpp
index 48b7d80f7..193b098e1 100644
--- a/fs_mgr/libsnapshot/snapshot_writer.cpp
+++ b/fs_mgr/libsnapshot/snapshot_writer.cpp
@@ -165,7 +165,7 @@ bool OnlineKernelSnapshotWriter::Finalize() {
 bool OnlineKernelSnapshotWriter::EmitRawBlocks(uint64_t new_block_start, const void* data,
                                               size_t size) {
   uint64_t offset = new_block_start * options_.block_size;
-  if (lseek(snapshot_fd_.get(), offset, SEEK_SET) < 0) {
+  if (lseek64(snapshot_fd_.get(), offset, SEEK_SET) < 0) {
     PLOG(ERROR) << "EmitRawBlocks lseek to offset " << offset;
     return false;
   }
diff --git a/fs_mgr/libsnapshot/snapuserd/user-space-merge/snapuserd_core.cpp b/fs_mgr/libsnapshot/snapuserd/
        user-space-merge/snapuserd_core.cpp
index 718c13ce0..ea986420c 100644
--- a/fs_mgr/libsnapshot/snapuserd/user-space-merge/snapuserd_core.cpp
+++ b/fs_mgr/libsnapshot/snapuserd/user-space-merge/snapuserd_core.cpp
@@ -401,7 +401,7 @@ bool SnapshotHandler::ReadBlocksAsync(const std::string& dm_block_device,
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
        struct iovec* iovec_ptr = vecs[i].get();

-       io_uring_prep_read(sqe, fd.get(), iovec_ptr->iov_base, iovec_ptr->iov_len, offset);
+       io_uring_prep_rw(IORING_OP_READ, sqe, fd.get(), iovec_ptr->iov_base, iovec_ptr->iov_len, offset);
        sqe->flags |= IOSQE_ASYNC;
        offset += bs;
    }
diff --git a/fs_mgr/libsnapshot/snapuserd/user-space-merge/snapuserd_merge.cpp b/fs_mgr/libsnapshot/snapuserd/
        user-space-merge/snapuserd_merge.cpp
index 63f47d6f0..c9301e740 100644
--- a/fs_mgr/libsnapshot/snapuserd/user-space-merge/snapuserd_merge.cpp
+++ b/fs_mgr/libsnapshot/snapuserd/user-space-merge/snapuserd_merge.cpp
@@ -118,7 +118,7 @@ bool Worker::MergeReplaceZeroOps() {
    size_t io_size = linear_blocks * BLOCK_SZ;

    // Merge - Write the contents back to base device
-   int ret = TEMP_FAILURE_RETRY(pwrite(base_path_merge_fd_.get(), bufsink_.GetPayloadBufPtr(),
+   int ret = TEMP_FAILURE_RETRY(pwrite64(base_path_merge_fd_.get(), bufsink_.GetPayloadBufPtr(),
                                        io_size, source_offset));
    if (ret < 0 || ret != io_size) {
        SNAP_LOG(ERROR)
@@ -219,7 +219,7 @@ bool Worker::MergeOrderedOpsAsync() {
            return false;
        }

-       io_uring_prep_write(sqe, base_path_merge_fd_.get(),
+       io_uring_prep_rw(IORING_OP_WRITE, sqe, base_path_merge_fd_.get(),
                            (char*)read_ahead_buffer + offset, io_size, source_offset);

        offset += io_size;
@@ -392,7 +392,7 @@ bool Worker::MergeOrderedOps() {
        // Write to the base device. Data is already in the RA buffer. Note
        // that XOR ops is already handled by the RA thread. We just write
        // the contents out.
-       int ret = TEMP_FAILURE_RETRY(pwrite(base_path_merge_fd_.get(),
+       int ret = TEMP_FAILURE_RETRY(pwrite64(base_path_merge_fd_.get(),
                                            (char*)read_ahead_buffer + offset, io_size,
                                            source_offset));
        if (ret < 0 || ret != io_size) {
diff --git a/fs_mgr/libsnapshot/snapuserd/user-space-merge/snapuserd_readahead.cpp b/fs_mgr/libsnapshot/
        snapuserd/user-space-merge/snapuserd_readahead.cpp
index b9e4255b2..c8dc89f72 100644
--- a/fs_mgr/libsnapshot/snapuserd/user-space-merge/snapuserd_readahead.cpp
+++ b/fs_mgr/libsnapshot/snapuserd/user-space-merge/snapuserd_readahead.cpp
@@ -282,7 +282,7 @@ bool ReadAhead::ReadAheadAsyncIO() {
            return false;
        }

-       io_uring_prep_read(sqe, backing_store_fd_.get(),
+       io_uring_prep_rw(IORING_OP_READ, sqe, backing_store_fd_.get(),
                           (char*)ra_temp_buffer_.get() + buffer_offset, io_size,
                           source_offset);
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5.4 修改分区表，OTA 升级后系统启动失败

5.4.1 问题描述

修改 sys_partition 分区表，OTA 升级后系统启动失败。

5.4.2 问题分析

从串口 log 看，是找不到 Reserve0 分区，导致安卓系统启动失败。

```
[2023-07-12 14:45:04] [    4.419811][   T66]  mmcblk0: p1 p2 p3 p4 p5 p6 p7 p8 p9 p10 p11 p12 p13 p14 p15 p16 p17 p18
    p19 p20 p21 p22 p23 p24 p25 p26 p27
[2023-07-12 14:45:04] [    4.423962][  T108] sunxi-mmc 4021000.sdmmc: detmode:manually by software
[2023-07-12 14:45:04] [    4.434496][   T66] mmcblk0rpmb: mmc0:0001 A3A561 16.0 MiB, chardev (235:0)
[2023-07-12 14:45:04] [    4.441213][    C0] sunxi-mmc 4021000.sdmmc: smc 2 p1 err, cmd 52, RTO !!
[2023-07-12 14:45:04] [    4.456631][    C0] sunxi-mmc 4021000.sdmmc: smc 2 p1 err, cmd 52, RTO !!
[2023-07-12 14:45:04] [    4.464269][   T89] sunxi-mmc 4021000.sdmmc: sdc set ios:clk 400000Hz bm PP pm ON vdd 21
    width 1 timing LEGACY(SDR12) dt B
[2023-07-12 14:45:04] [    4.479643][   T89] sunxi-mmc 4021000.sdmmc: sdc set ios:clk 400000Hz bm PP pm ON vdd 21
    width 1 timing LEGACY(SDR12) dt B
[2023-07-12 14:45:04] [    4.494129][    C0] sunxi-mmc 4021000.sdmmc: smc 2 p1 err, cmd 5, RTO !!
[2023-07-12 14:45:04] [    4.502484][    C0] sunxi-mmc 4021000.sdmmc: smc 2 p1 err, cmd 5, RTO !!
[2023-07-12 14:45:04] [    4.510836][    C0] sunxi-mmc 4021000.sdmmc: smc 2 p1 err, cmd 5, RTO !!
[2023-07-12 14:45:04] [    4.519189][    C0] sunxi-mmc 4021000.sdmmc: smc 2 p1 err, cmd 5, RTO !!
[2023-07-12 14:45:04] [    4.526729][   T89] sunxi-mmc 4021000.sdmmc: sdc set ios:clk 0Hz bm PP pm OFF vdd 0 width 1
    timing LEGACY(SDR12) dt B
[2023-07-12 14:45:04] [   12.552955][    T1] init: Wait for partitions returned after 10001ms
[2023-07-12 14:45:12] [   12.560190][    T1] init: bool android::init::BlockDevInitializer::InitDevices(std::set<std::string>):
    partition(s) not found after polling timeout: Reserve0
[2023-07-12 14:45:12] [   12.576167][    T1] init: Failed to mount required partitions early ...
[2023-07-12 14:45:12] [   12.584116][    T1] init: InitFatalReboot: signal 6
[2023-07-12 14:45:12] [   12.590435][    T1] init: #00 pc 000000000030a36c  /init
[2023-07-12 14:45:12] [   12.596405][    T1] init: Reboot ending, jumping to kernel
[2023-07-12 14:45:12] [   12.602567][    T1] reboot_callback(): store flag 'bootloader' (0x5f) in RTC General-Purpose-
    Register
[2023-07-12 14:45:12] [   12.618285][    T1] sunxi-mmc 4022000.sdmmc: sdc set ios:clk 0Hz bm PP pm OFF vdd 0 width 1
    timing LEGACY(SDR12) dt B
[2023-07-12 14:45:12] [   12.631616][    T1] sunxi-twi 2503400.twi5: xfer completed, shutdown twi directly
[2023-07-12 14:45:12] [   12.640015][    T1] sunxi-twi 2503400.twi5: shutdown twi after waiting for the transfer to complete
[2023-07-12 14:45:12] [   12.650161][    T1] sunxi-twi 2503000.twi4: xfer completed, shutdown twi directly
[2023-07-12 14:45:12] [   12.658548][    T1] sunxi-twi 2503000.twi4: shutdown twi after waiting for the transfer to complete
[2023-07-12 14:45:12] [   12.668694][    T1] sunxi-twi 2502c00.twi3: xfer completed, shutdown twi directly
[2023-07-12 14:45:12] [   12.677086][    T1] sunxi-twi 2502c00.twi3: shutdown twi after waiting for the transfer to complete
[2023-07-12 14:45:12] [   12.687232][    T1] sunxi-twi 2502800.twi2: xfer completed, shutdown twi directly
[2023-07-12 14:45:12] [   12.695625][    T1] sunxi-twi 2502800.twi2: shutdown twi after waiting for the transfer to complete
[2023-07-12 14:45:12] [   12.705768][    T1] sunxi-twi 2502000.twi0: xfer completed, shutdown twi directly
[2023-07-12 14:45:12] [   12.714161][    T1] sunxi-twi 2502000.twi0: shutdown twi after waiting for the transfer to complete
[2023-07-12 14:45:12] [   13.000656][    T1] [DISP WRN]: [de_smbl_enable_ahb_read:0544]: de_smbl_enable_ahb_read=0
[2023-07-12 14:45:12] [   13.011974][    T1] [DISP WRN]: [de_dci_enable_ahb_read:0705]: de_dci_enable_ahb_read=0
[2023-07-12 14:45:12] [   13.036261][    T1] sunxi-twi 7081800.s_twi1: xfer completed, shutdown twi directly
[2023-07-12 14:45:12] [   13.044850][    T1] sunxi-twi 7081800.s_twi1: shutdown twi after waiting for the transfer to
    complete
[2023-07-12 14:45:12] [   13.055364][    T1] sunxi-twi 7081400.s_twi0: xfer completed, shutdown twi directly
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
[2023-07-12 14:45:12] [   13.063970][    T1] sunxi-twi 7081400.s_twi0: shutdown twi after waiting for the transfer to
      complete
[2023-07-12 14:45:12] [   13.075090][    T1] reboot: Restarting system with command 'bootloader'
```

5.4.3 问题解决方案

第一次量产时就要将分区的数目和大小确定清楚，杜绝后续升级调整分区数目及其大小的想法，OTA 不能改变分区数目和分区的大小。

5.5 修改 fstab 文件，OTA 升级后系统启动失败

5.5.1 问题描述

修改 fstab 文件，改变 data 分区的挂载参数，导致 OTA 升级后，data 分区读写异常，系统启动失败。

5.5.2 问题分析

uboot 打开串口 log，从串口 log 看，是设置/data/bootchart 目录加密策略失败，导致安卓系统启动失败，进入 recovery 系统

```
[   16.888560][    T1] init: Failed to set encryption policy of /data/bootchart to 1bc3a10dee74d77fd8de9c68dae3ba95 v2
      modes 1/4 flags 0xa: The directory already has a different encryption policy.
[   16.907812][    T1] init: Setting 1bc3a10dee74d77fd8de9c68dae3ba95 policy on /data/bootchart failed!
[   16.918066][    T1] init: Rebooting into recovery
```

5.5.3 问题解决方案

第一次量产时就要将 fstab 文件各个分区挂载参数确定清楚，杜绝后续升级调整分区挂载参数的想法，OTA 不能改变分区挂载参数。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6 FAQ

6.1 ota 更新 SettingsProvider 数据库

可以在 ota 升级的时候，更新 SettingsProvider 的 SETTINGS_VERSION，这样就可以 ota 之后，做 SettingsProvider 更新操作，如下面更新系统语言的值。

```
diff --git a/packages/SettingsProvider/src/com/android/providers/settings/SettingsProvider.java b/packages/
        SettingsProvider/src/com/android/providers/settings/SettingsProvider.java
index 9ac775801382..3fdadbfeee9a 100644
--- a/packages/SettingsProvider/src/com/android/providers/settings/SettingsProvider.java
+++ b/packages/SettingsProvider/src/com/android/providers/settings/SettingsProvider.java
@@ -3652,7 +3652,7 @@ public class SettingsProvider extends ContentProvider {
        }

        private final class UpgradeController {
-           private static final int SETTINGS_VERSION = 212;
+           private static final int SETTINGS_VERSION = 213;

            private final int mUserId;

@@ -5568,6 +5568,47 @@ public class SettingsProvider extends ContentProvider {
                    currentVersion = 212;
                }

+               if (currentVersion == 212) {
+                   final SettingsState systemSettings = getSystemSettingsLocked(userId);
+                   Slog.d(LOG_TAG, "update settingsprovider after ota");
+                   systemSettings.insertSettingOverrideableByRestoreLocked(
+                       Settings.System.SYSTEM_LOCALES,
+                       SystemProperties.get("ro.product.locale", "en-US"),
+                       null,
+                       true,
+                       SettingsState.SYSTEM_PACKAGE_NAME);
+                   currentVersion = 213;
+               }
+
                // vXXX: Add new settings above this point.

                if (currentVersion != newVersion) {
```

6.2 如何支持 ota 升级到旧版本系统

如果需要支持 OTA 升级到旧版本系统，可以去掉 update_engine 升级程序中对时间戳的检查。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
// system/update_engine/payload_consumer/delta_performer.cc

注释掉对时间戳的检查的代码。
/*
 ErrorCode error_code = CheckTimestampError();
 if (error_code != ErrorCode::kSuccess) {
  if (error_code == ErrorCode::kPayloadTimestampError) {
   if (!hardware_->AllowDowngrade()) {
    return ErrorCode::kPayloadTimestampError;
   }
   LOG(INFO) << "The current OS build allows downgrade, continuing to apply"
             " the payload with an older timestamp.";
  } else {
   LOG(ERROR) << "Timestamp check returned "
            << utils::ErrorCodeToString(error_code);
   return error_code;
  }
 }
*/
```

6.3 升级注意事项

6.3.1 OTA 不能改变分区数目及其大小以及 fstab 参数

update_engine 升级程序没有能力对现有分区表进行调整，所以第一次量产时就要将分区的数目和大小确定清楚，杜绝后续升级调整分区数目及其大小的想法，OTA 不能改变分区数目和分区的大小。

修改分区数目和分区的大小，会导致 OTA 升级之后，系统起不来。

修改 fstab 挂载参数，如修改 data 分区的挂载参数，会导致 OTA 升级之后，data 分区挂载失败，从而进入 recovery 系统。

6.3.2 Recovery 升级

注意事项:

```
recovery模式下：

1. 因无权限读写data分区，所以没办法创建snapshot，没办法进行差分包升级。
2. 如果super分区size小于镜像两倍，完整包升级会擦除原有的super分区。
3. 如果super分区size大于镜像两倍，完整包升级不会擦除原有的super分区。
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6.4 制作 OTA 包常见问题和注意事项

1. TargetFile 和固件是否匹配的区分方法：在 Android 设备执行 adb pull /system/build.prop，会得到这个固件的 build.prop 文件，对于 TargetFile，解压出来查看 SYSTEM/build.prop，对比这两个 build.prop，如果一致，表示这个固件和 TargetFile 是匹配的。
2. metadata 匹配失败必须获取对应的 payload_properties.txt，每个升级包内都是不一样的。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2025 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、全志科技、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。